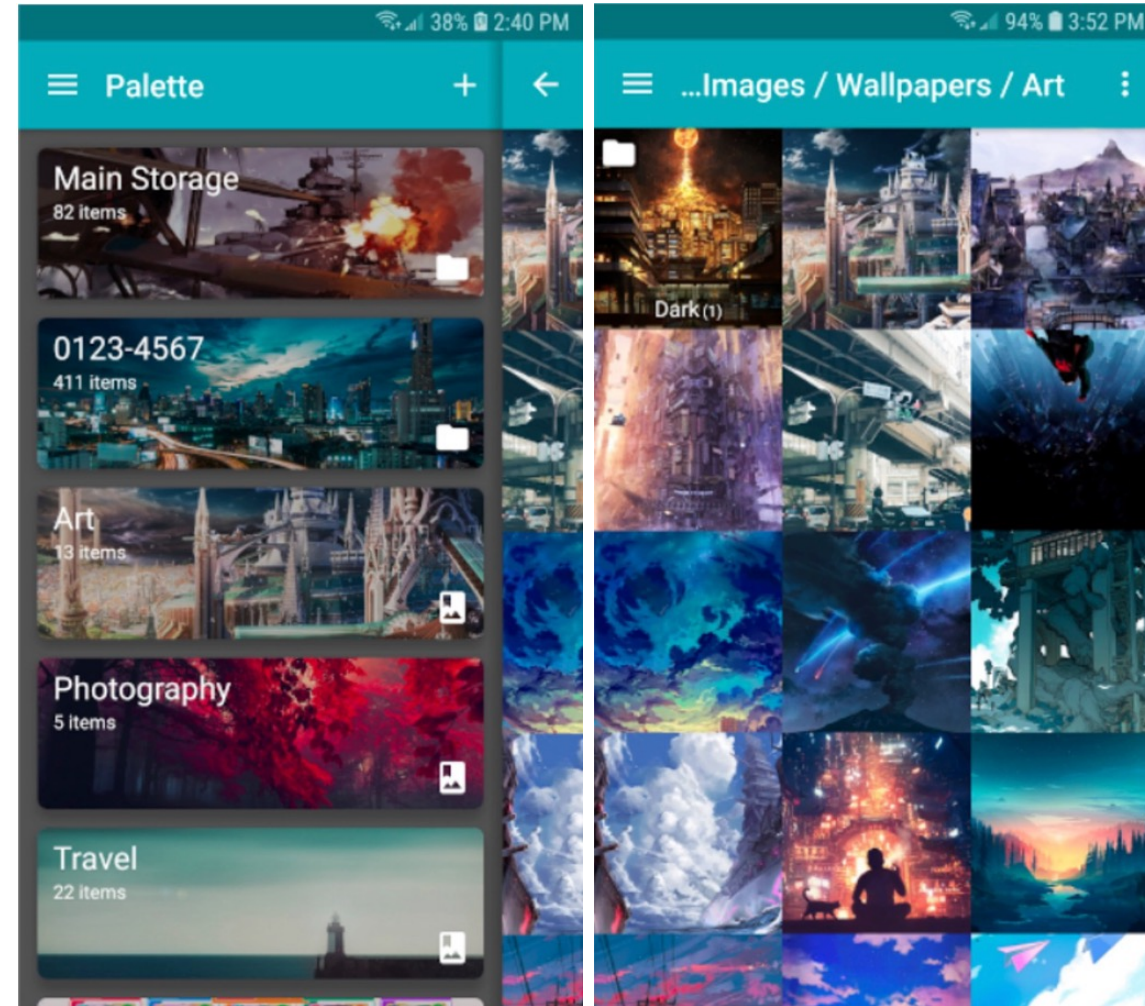




Android Basics

Activities

- **Activity:** A **component** that represents one screen or entry point in an Android app
 - Each screen shown is likely backed by one activity
- In Kotlin, an Activity is usually a class, such as MainActivity
- The Activity loads a layout and responds to user action



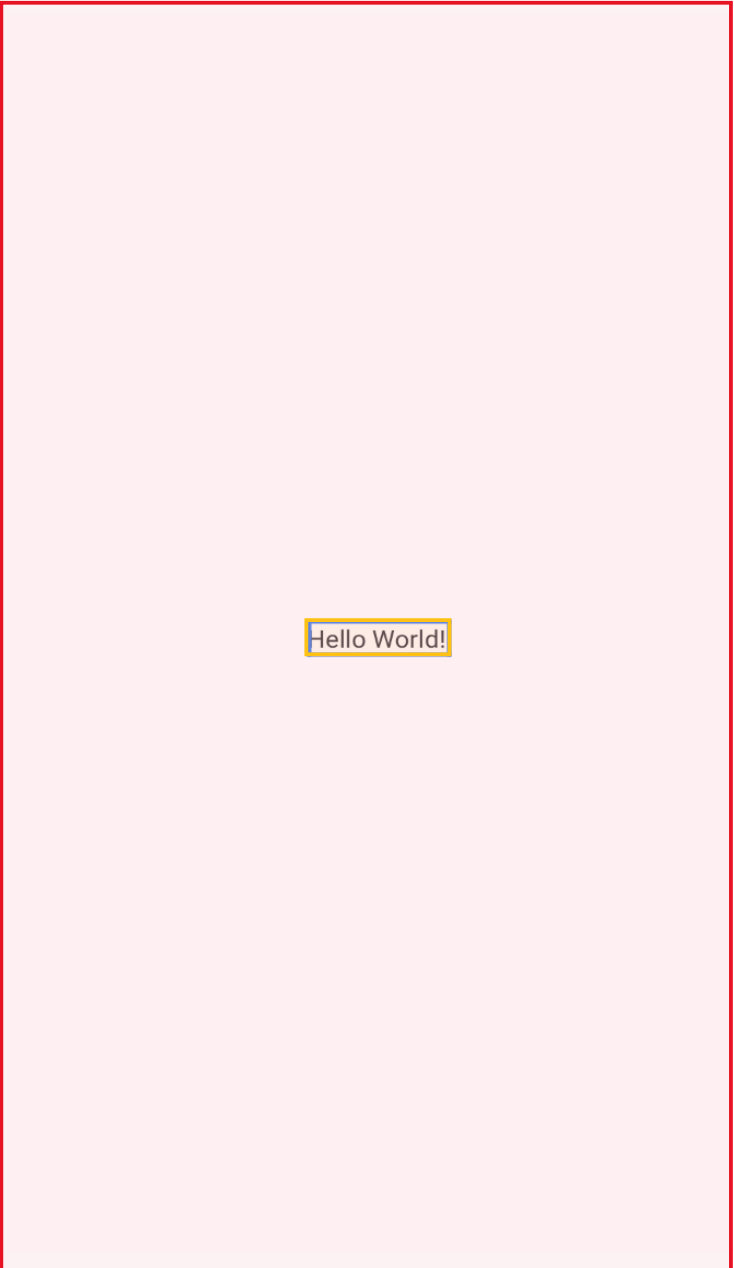
Layout File

- There are two parts to every activity:
- 1) The Activity's **layout file**

XML that defines UI widgets on the screen

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello World!"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
```



Hello World!

Activity Class

- 2) The Activity class (**Android/Kotlin** code) that controls the screen behaviour
 - This is where we define **behaviour** and **logic** for the screen
 - E.g., do something when a Button is clicked
- When you start up an app or navigate to a new activity, the **Android OS calls onCreate**

```
class MainActivity : AppCompatActivity() { 2 Usages
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.activity_main)
    }
}
```

“setContentView” connects the activity to its layout file

Activities: Connecting the UI to Code

- Now that the Kotlin class is associated with a layout file, how do we refer to the layout's UI widgets from our code?
 - Use **findViewById**

```
class MainActivity : AppCompatActivity() { 2 Usages
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val helloTextView = findViewById<TextView>(R.id.hello_text_view)
        helloTextView.text = "World, Hello!"
    }
}
```

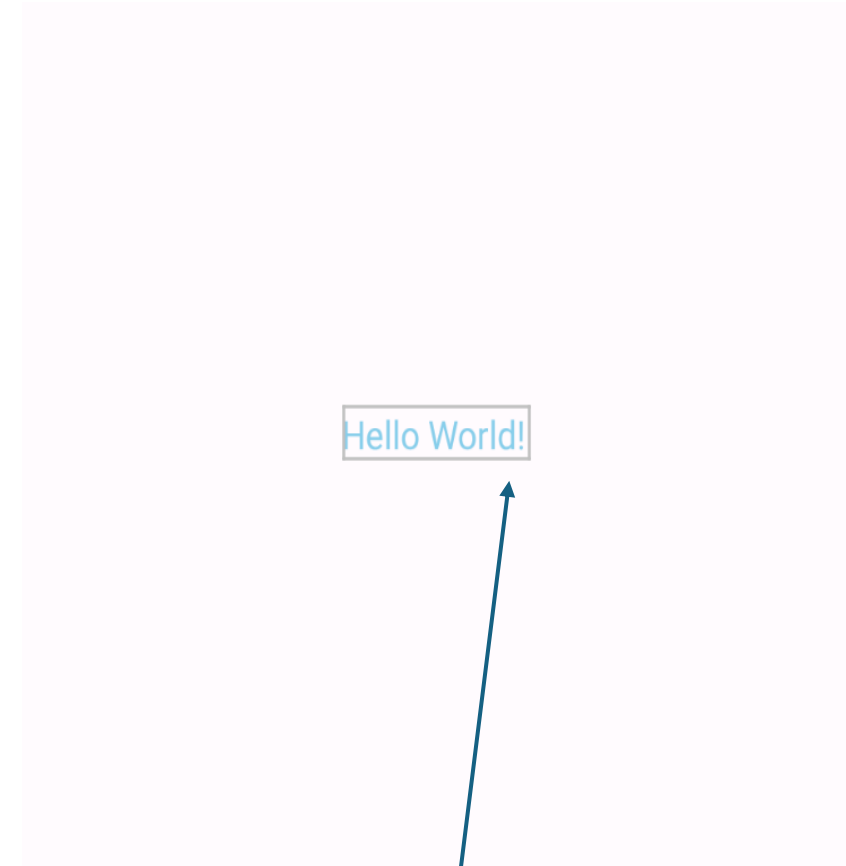
```
<TextView
    android:id="@+id/hello_text_view"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hello World!"
```

activity_main.xml

World, Hello!

Views

- A View is a UI widget, such as TextView, ListView, or Button
- XML attributes control how a view appears in the layout
 - Ex.: id, text, width, height, textSize, textColor
- If Kotlin needs to access a View, gives the View an ID



```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="black">#FF000000</color>
    <color name="white">#FFFFFFF</color>
     <color name="sky_blue">#FF87CEEB</color>
</resources>
```

colors.xml

```
<TextView
    android:id="@+id/hello_text_view"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hello World!"
    android:textColor="@color/sky_blue"
```

activity_main.xml

ViewGroups

- A ViewGroup is a layout container that holds and positions other Views
- A ViewGroup controls where their child view appears on the screen
 - Ex.:
 - LinearLayout
 - ConstraintLayout
 - FrameLayout
- The root element of activity_main.xml is usually a ViewGroup


```
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">
```

```
<TextView
```

```
    android:id="@+id/hello_text_view"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hello World!"
    android:textColor="@color/sky_blue"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent" />
```

```
<TextView
```

```
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hi CMPUT 301!"
    android:textSize="36sp"
    tools:ignore="HardcodedText"
    tools:layout_editor_absoluteX="82dp"
    tools:layout_editor_absoluteY="297dp" />
```

Hi CMPUT 301!

Hello World!



Examples of ViewGroups

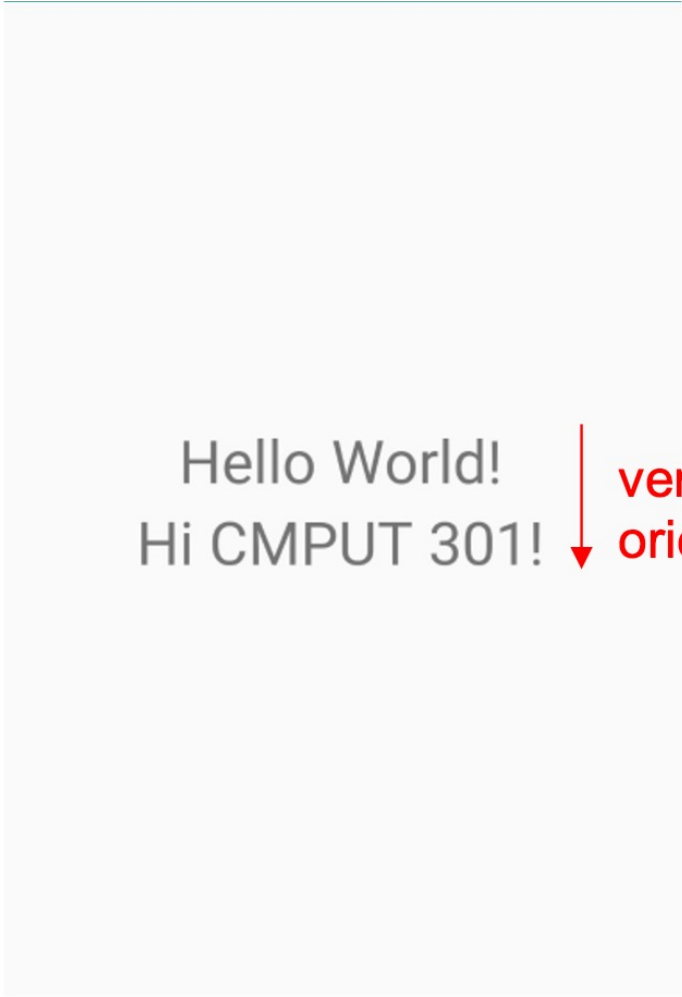
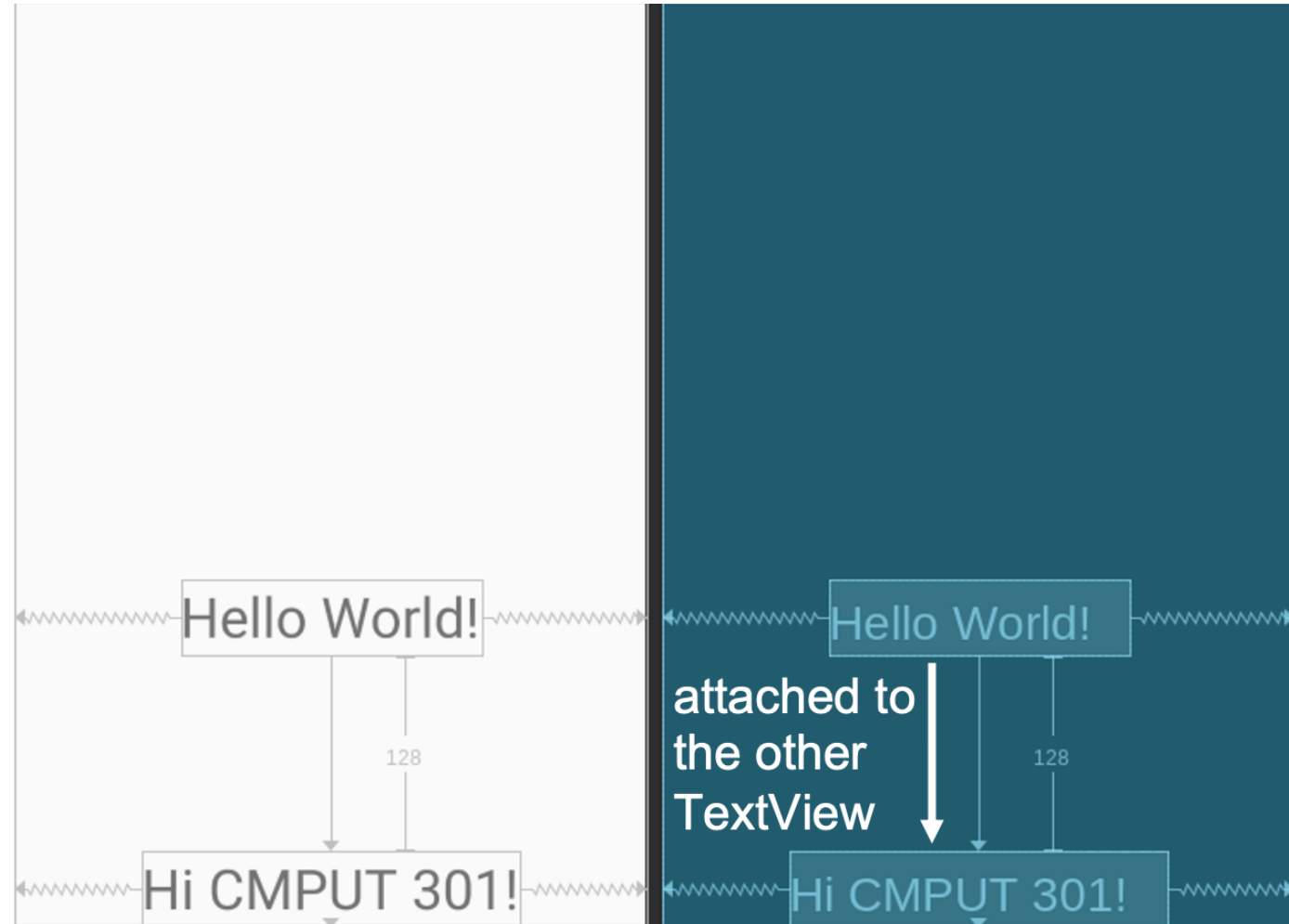


Diagram illustrating a **LinearLayout**. It shows a light gray rectangular container. Inside, the text "Hello World!" is positioned above the text "Hi CMPUT 301!". A red arrow points downwards from the first line of text to the second, with the text "vertical orientation" written in red next to it.

Hello World!
Hi CMPUT 301! ↓ vertical orientation

LinearLayout positions children one after another (horizontally or vertically)



ConstraintLayout positions children by attaching them to the edges of other views

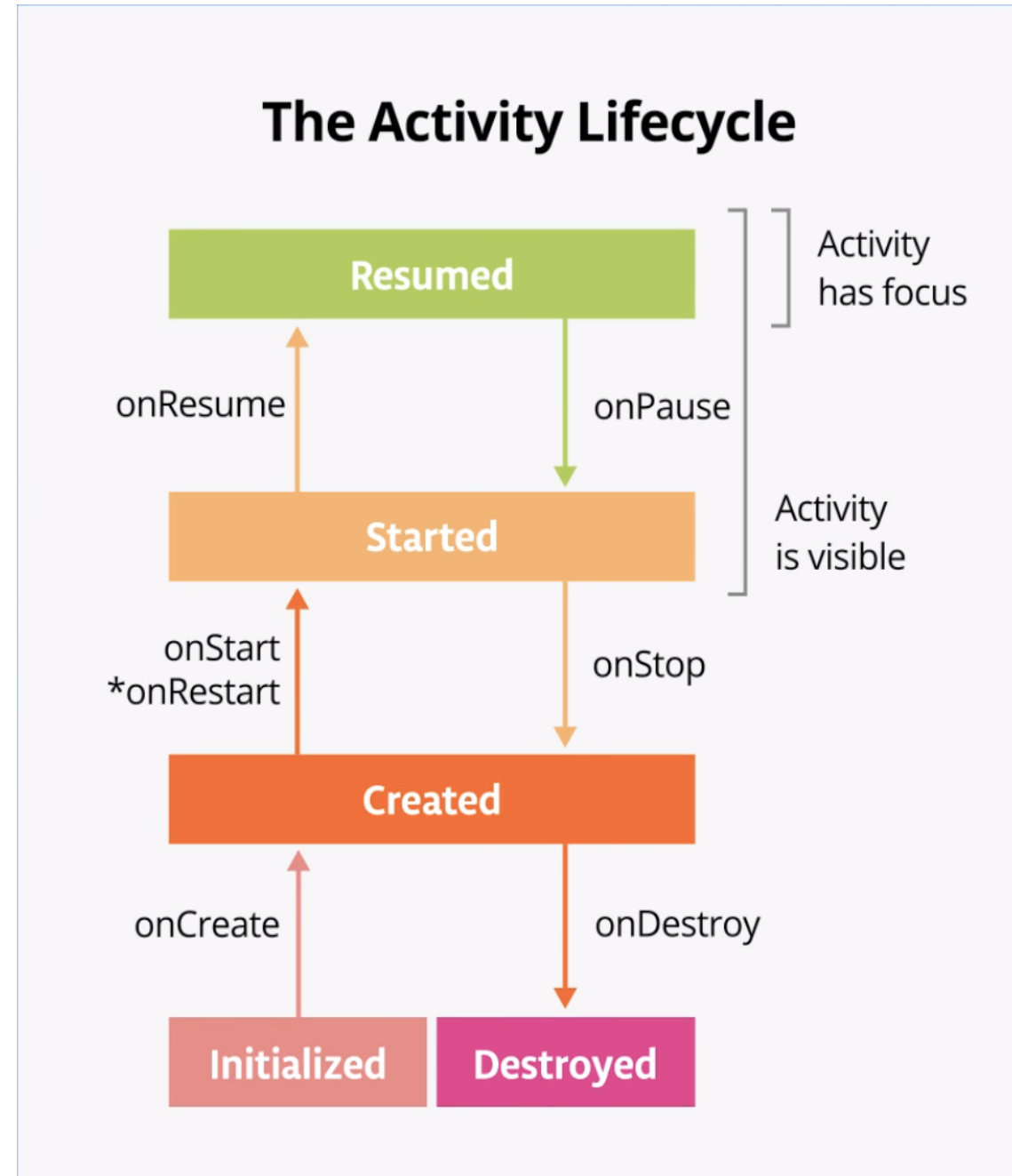
Activity Lifecycle

- **onCreate()** is called when Android creates an Activity
- It is commonly used to:
 - Load the layout
 - Connect Kotlin code to Views
 - Initialize screen state
 - Set up event listeners
- Think of it as the Activity's setup method: not exactly the same as `main()`, but often the first method we work with

```
class MainActivity : AppCompatActivity() { 2 Usages
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.activity_main)
    }
}
```

Activity Lifecycle

- Android calls lifecycle methods as an Activity changes state
- Common lifecycle methods:
 - onCreate() – set up Activity
 - onStart() - Activity becomes visible
 - onResume() – Activity becomes interactive
 - onPause() or onStop() – Activity is leaving the screen
 - onDestroy() – Activity is being destroyed
- We usually begin with onCreate(), because that is where the Activity is first set up.



Activity Lifecycle

- **Never** call these methods yourself
 - Leave that to **Android**
- Our job is to override lifecycle methods when we need to add our own logic

```
class MainActivity : AppCompatActivity() { 2 Usages
    // override, then add additional logic
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        // Add setup logic here
    }

    override fun onStart() {
        super.onStart()
        // Add logic for when the Activity becomes visible
    }

    override fun onResume() {
        super.onResume()
        // Add logic for when the Activity becomes interactive
    }

    override fun onPause() {
        super.onPause()
        // Add logic for when the Activity is partially leaving the screen
    }

    override fun onStop() {
        super.onStop()
        // Add logic for when the Activity is no longer visible
    }

    override fun onDestroy() {
        super.onDestroy()
        // Add cleanup logic here
    }
}
```

Good to Know for Assignment 1

- How to do something when a view is clicked (View.setOnClickListener)
 - Use of **Intents** to navigate to another activity
 - How to **send data between activities** (send it through Intent)
 - **Android Manifest**
 - How to display a list of items (**ListView**, **ListAdapter**)
-
- These concepts will be covered in the lab

References

Android Icon - Made by Freepik from www.flaticon.com

Activity Wallpaper Art – by J. Mah, CMPUT 301, 2019

Android Lifecycle Diagram – by Android Developers from <https://developer.android.com/codelabs/basic-android-kotlin-compose-activity-lifecycle#2>